

Multi Layer Perceptron

Week 11

D4 Robotics Engineering Study Program
Department of Electrical Engineering
Politeknik Negeri Batam

Machine Learning

Non Linear Problem

01

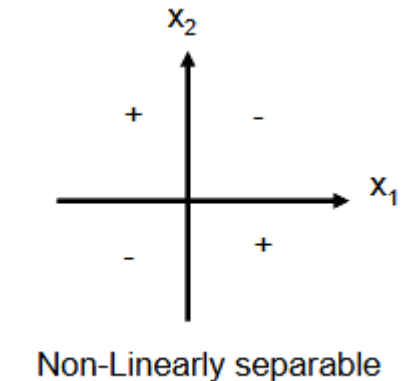
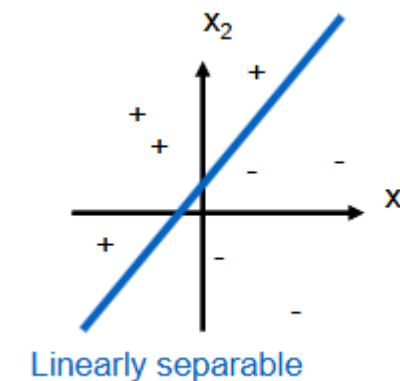
Review of Perceptron-Linear Problem

Perceptron mampu menyelesaikan beberapa fungsi, misalnya AND, OR, NOT. Tetapi fungsi seperti Non-Linear yang terdapat pada XOR tidak dapat diselesaikan.

AND Gate - Linearly Separable



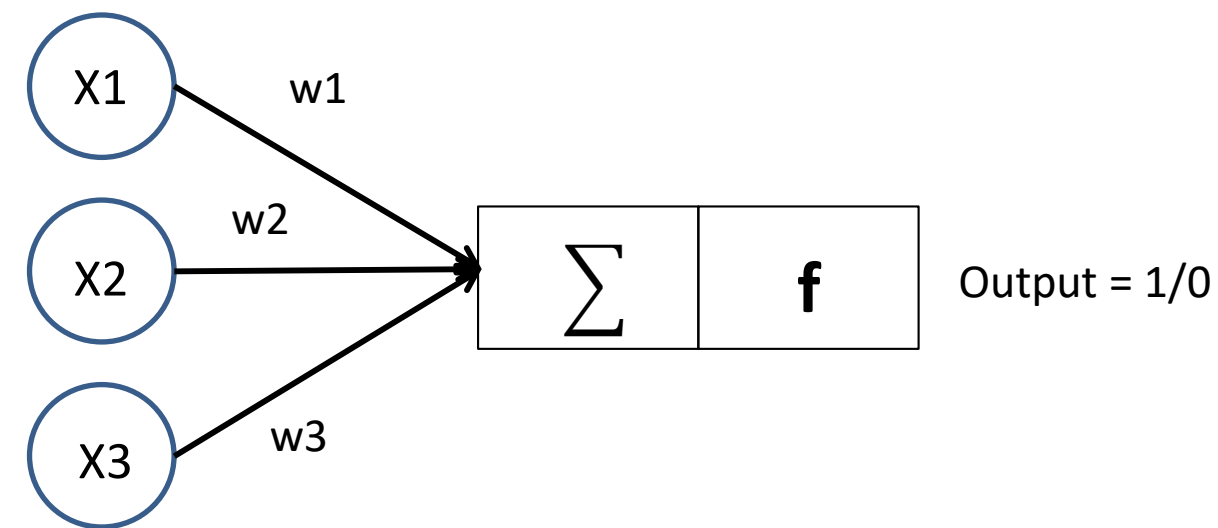
XOR Gate - Non-linearly Separable



Single Layer Perceptron

Single Layer Perceptron adalah model neural network paling sederhana yang terdiri dari:

- Satu layer input
- Satu layer output
- Tidak ada hidden layer



Karakteristik Utama SLP:

- **Linear classifier:** Hanya dapat memisahkan data linear
- **Binary output:** Menghasilkan output 0 atau 1
- **Step function:** Menggunakan fungsi aktivasi tangga
- **Supervised learning**

Limitations of Single Layer Perceptron

✗ Masalah Utama: XOR Problem

Single layer perceptron **TIDAK DAPAT** menyelesaikan masalah yang tidak linearly separable

✓ Bisa Diselesaikan:

A	B	AND
0	0	0
0	1	0
1	0	0
1	1	1

Linearly Separable - Bisa dipisah dengan garis lurus

✗ Tidak Bisa Diselesaikan:

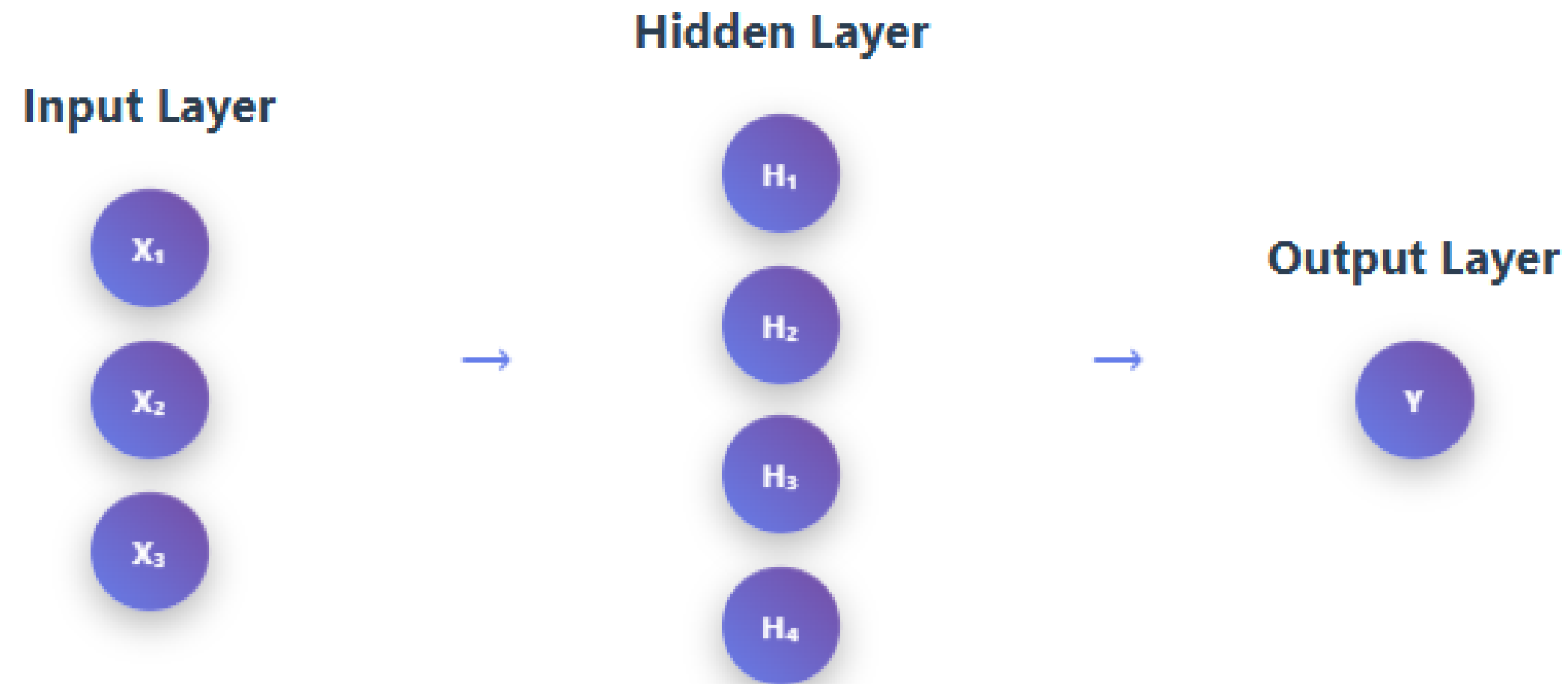
A	B	XOR
0	0	0
0	1	1
1	0	1
1	1	0

Non-linearly Separable - Tidak bisa dipisah dengan garis lurus

Non-Linear Problem

02

Multi Layer Perceptron



Deep Learning & Neural Networks

Memahami Non-Linear Problem Solving dengan Backpropagation

Multi Layer Perceptron

Multi-Layer Perceptron adalah jenis neural network yang terdiri dari **beberapa layer** neuron yang saling terhubung.

Karakteristik Utama MLP:

Feedforward Network: Informasi mengalir dari input ke output

Multiple Layers: Input layer, hidden layer(s), output layer

Non-linear Activation: Menggunakan fungsi aktivasi non-linear

Supervised Learning: Belajar dari data training dengan target

Struktur Dasar MLP:

Input $\rightarrow$ Hidden Layer(s) $\rightarrow$ Output

$$y = f(W_2 \cdot \sigma(W_1 \cdot x + b_1) + b_2)$$

Non-Linear Problem

Masalah non-linear adalah masalah yang **tidak dapat diselesaikan dengan garis lurus** (linear separator).

Mengapa Perceptron Tunggal Gagal?

Contoh: Masalah XOR

Input A	Input B	Output (A XOR B)
0	0	0
0	1	1
1	0	1
1	1	0

Problem: Tidak ada garis lurus yang dapat memisahkan output 0 dan 1!

Contoh Masalah Non-Linear Lainnya:

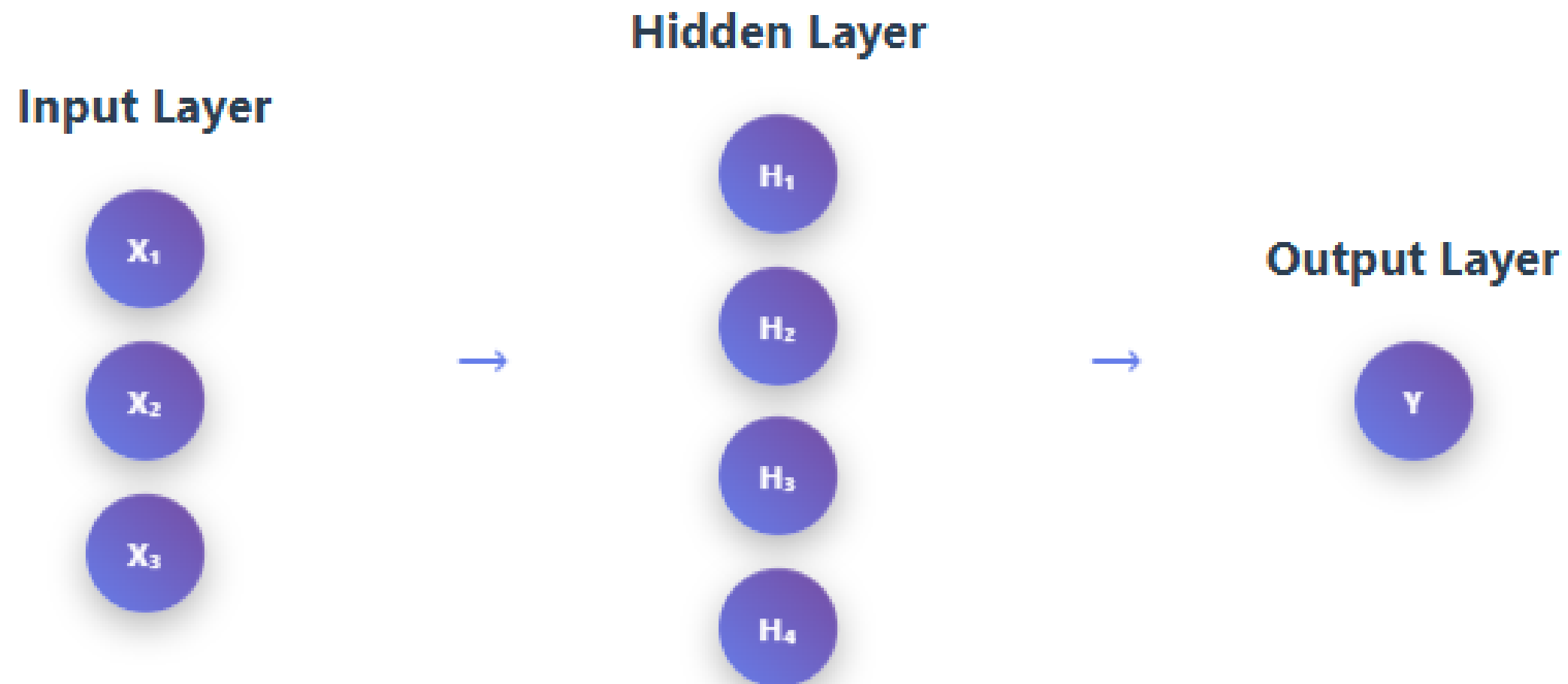
Pattern Recognition: Mengenali wajah, objek, tulisan tangan

Function Approximation: $\sin(x)$, x^2 , polynomial functions

Classification: Data yang tidak linearly separable

Time Series: Prediksi cuaca, harga saham

Arsitektur Multi Layer Perceptron

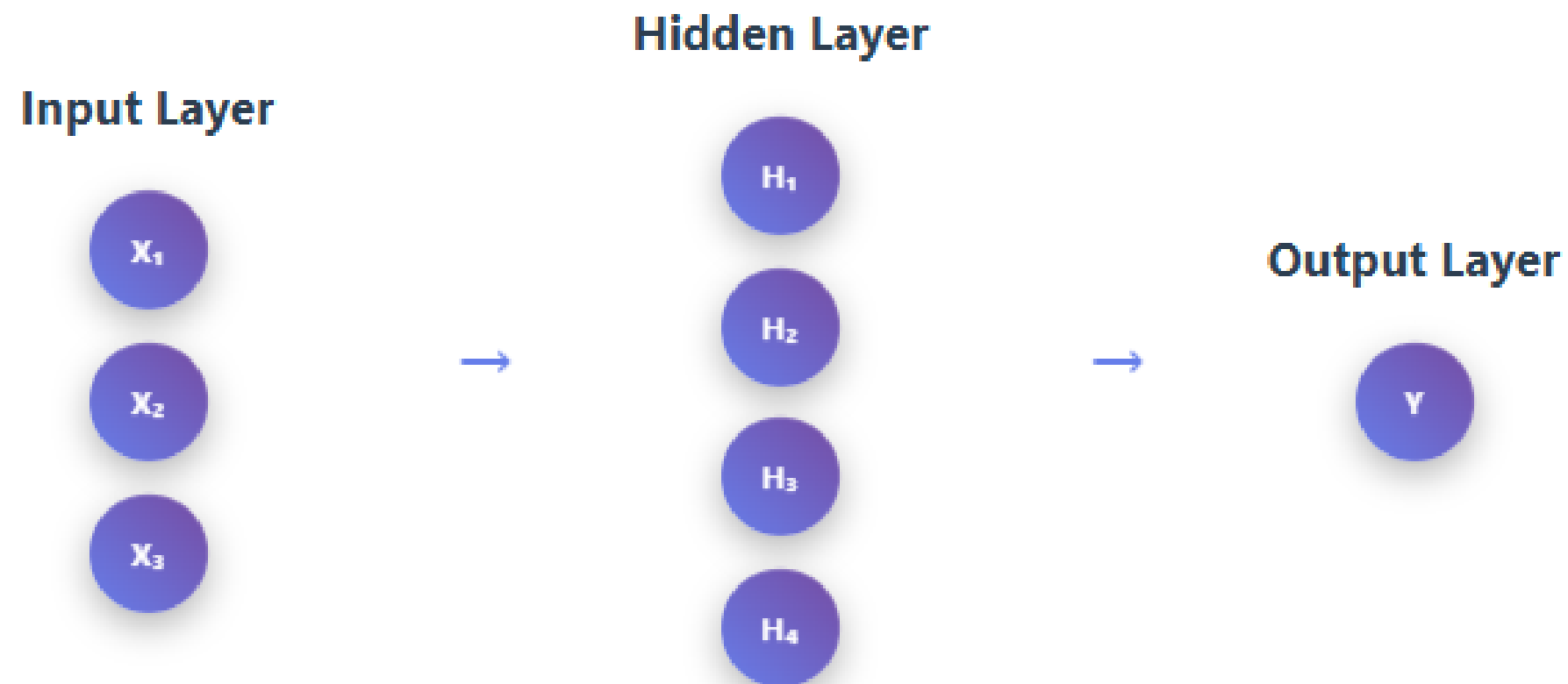


Solusi MLP:

Hidden layers + Non-linear activation functions

= Kemampuan memproksimasi fungsi non-linear kompleks!

Arsitektur Multi Layer Perceptron



Komponen Arsitektur:

Input Layer: Menerima data input tanpa melakukan komputasi

Hidden Layer(s): Melakukan transformasi non-linear pada data

Output Layer: Menghasilkan prediksi final
Weights (W): Parameter yang dipelajari selama training

Bias (b): Threshold untuk aktivasi neuron

Rumus Forward Pass:

$$\text{Hidden Layer: } h = \sigma(W_1 \cdot x + b_1)$$

$$\text{Output Layer: } y = \sigma(W_2 \cdot h + b_2)$$

Konsep Dasar Neuron Buatan

Formula Matematika

$$y = f(\sum(w_i \times x_i) + b)$$

Dimana:

- x_i = input ke- i
- w_i = bobot ke- i
- b = bias
- f = activation function

Faktor-faktor:

- Cuaca cerah ($x_1 = 0.8$) dengan bobot penting ($w_1 = 0.7$)
- Punya waktu luang ($x_2 = 0.6$) dengan bobot ($w_2 = 0.5$)
- Ada teman ($x_3 = 0.9$) dengan bobot ($w_3 = 0.3$)
- Bias personal ($b = -0.2$) - Anda agak malas keluar rumah

Perhitungan:

$$\begin{aligned} \text{net} &= (0.8 \times 0.7) + (0.6 \times 0.5) + (0.9 \times 0.3) + (-0.2) \\ &= 0.56 + 0.30 + 0.27 - 0.20 \\ &= 0.93 \end{aligned}$$

$$\text{Keputusan} = \sigma(0.93) = 0.717 \text{ (71.7\% kemungkinan pergi piknik)}$$

Arsitektur Multi Layer Perceptron

Rumus Forward Pass:

Hidden Layer: $h = \sigma(W_1 \cdot x + b_1)$

Output Layer: $y = \sigma(W_2 \cdot h + b_2)$

Hidden Layer vs Output Layer

Hidden Layer

$$h = \sigma(W_1 \cdot x + b_1)$$

Proses:

Input x dari layer sebelumnya (atau input asli)

W_1 adalah matriks bobot yang menghubungkan setiap input ke setiap hidden neuron

b_1 adalah bias untuk setiap hidden neuron

σ mentransformasi hasil linear menjadi non-linear

Contoh dengan 3 input, 2 hidden neurons:

Matrix form:

$$\begin{bmatrix} h_1 \\ h_2 \end{bmatrix} = \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} + \begin{bmatrix} b_1 \\ b_2 \end{bmatrix}$$

Kemudian diaktivasi:

$$h_1 = \sigma(w_{11}x_1 + w_{12}x_2 + w_{13}x_3 + b_1)$$

$$h_2 = \sigma(w_{21}x_1 + w_{22}x_2 + w_{23}x_3 + b_2)$$

Arsitektur Multi Layer Perceptron

Hidden Layer vs Output Layer

Hidden Layer

$$h = \sigma(W_1 \cdot x + b_1)$$

Proses:

Input x dari layer sebelumnya (atau input asli)

W₁ adalah matriks bobot yang menghubungkan setiap input ke setiap hidden neuron

b₁ adalah bias untuk setiap hidden neuron

σ mentransformasi hasil linear menjadi non-linear

Contoh dengan 3 input, 2 hidden neurons:

Matrix form:

$$\begin{aligned} [h_1] &= [w_{11} \ w_{12} \ w_{13}] \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} + [b_1] \\ [h_2] &= [w_{21} \ w_{22} \ w_{23}] \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} + [b_2] \end{aligned}$$

Kemudian diaktivasi:

$$\begin{aligned} h_1 &= \sigma(w_{11}x_1 + w_{12}x_2 + w_{13}x_3 + b_1) \\ h_2 &= \sigma(w_{21}x_1 + w_{22}x_2 + w_{23}x_3 + b_2) \end{aligned}$$

Output Layer:

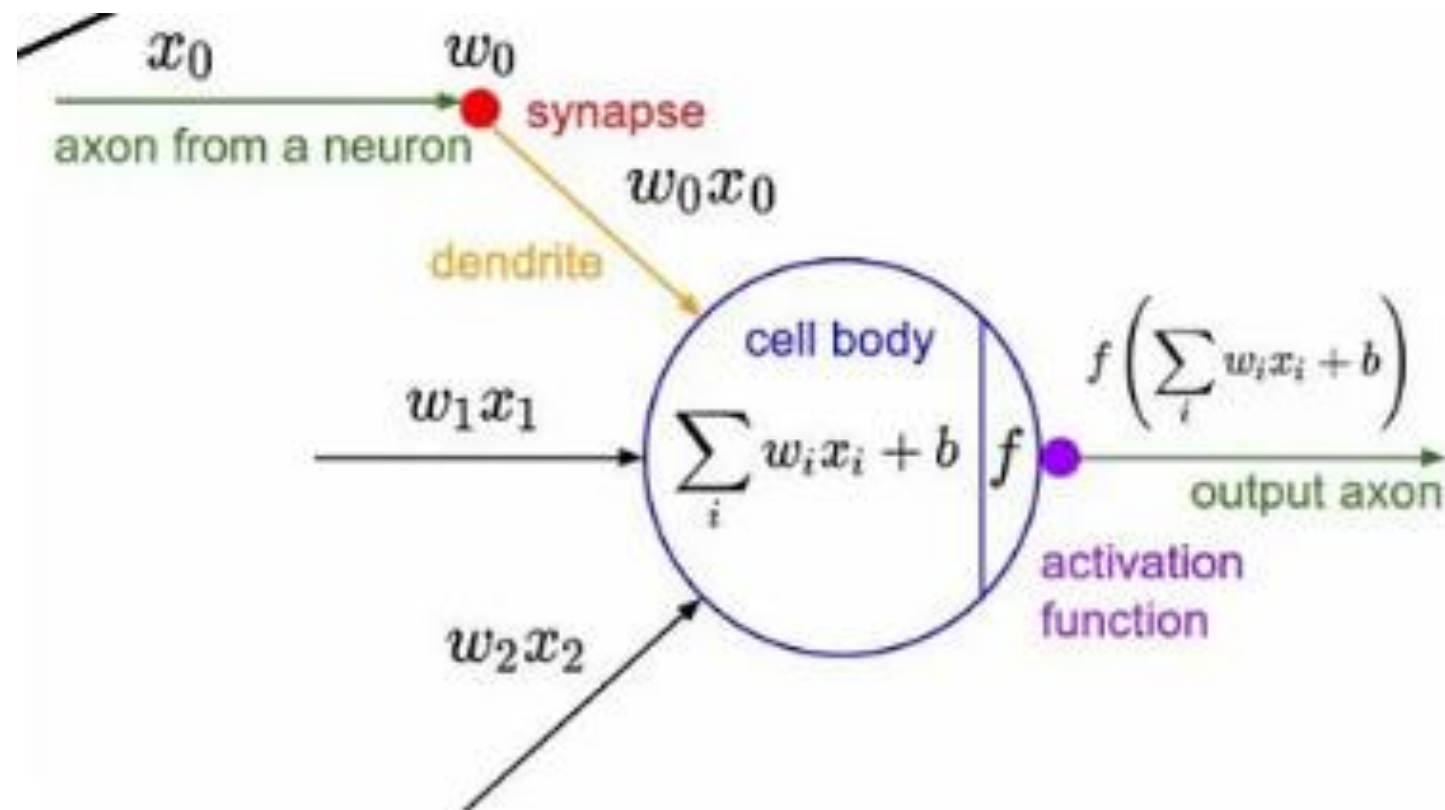
$$y = \sigma(W_2 \cdot h + b_2)$$

Proses sama, tapi:

- **Input h** adalah output dari hidden layer
- **W₂** menghubungkan hidden neurons ke output neurons
- **σ** di output bisa berbeda (misal: softmax untuk klasifikasi)

Activation Function - Moment Of Decision

Activation Function mengatur apakah neuron tersebut harus aktif atau tidak.



Activation Function untuk kasus **Linear**

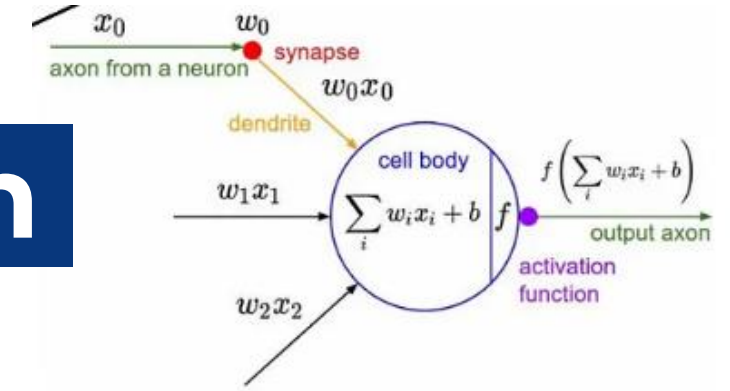
Step Function

Greater or equal to 1 = 1
Otherwise = 0

Step Function - Guru tegas:

- Skor ≥ 0 : 'MASUK!' (output = 1)
- Skor < 0 : 'KELUAR!' (output = 0)

Activation Function - Moment Of Decision



- **Sigmoid**

Sigmoid function dikenal juga dengan istilah logistic function, akan menghasilkan nilai pada rentang $[0-1]$, dirumuskan sebagai,

$$f(x) = \sigma(x) = \frac{1}{1 + e^{-x}}$$

- **Rectified Linear Unit (ReLU)**

Rectified Linear Unit (ReLU) function memiliki kelebihan dalam Network yang diinisiasi secara random, hanya 50% dari hidden layer yang akan di aktivasi. ReLU Function di rumuskan sebagai berikut,

$$f(x) = \max(0, x)$$

atau dalam bentuk rumus,
$$f(x) = \begin{cases} 0 & \text{for } x \leq 0 \\ x & \text{for } x > 0 \end{cases}$$

- **Softmax**

Softmax Funtion atau disebut juga softmax regression adalah bentuk logistic regression yang input valuenya di normalisasi kedalam probability distribution, dirumuskan sebagai,

$$f_i(\vec{x}) = \frac{e^{x_i}}{\sum_{j=1}^J e^{x_j}}$$

Keuntungan utama dari fungsi ini adalah mampu menangani beberapa kelas.

Baca Lebih Lanjut:

<https://medium.com/@cmukesh8688/activation-functions-sigmoid-tanh-relu-leaky-relu-softmax-50d3778dcea5>

Activation Function - Moment Of Decision

Mengapa Fungsi Aktivasi Penting?

- Tanpa Aktivasi Non-Linear: MLP = Linear model (tidak berguna)
- Dengan Aktivasi non-Linear MLP dapat mempelajari pola kompleks
- Gradient Flow: Memungkinkan backpropagation bekerja
- Decision Boundaries: Membuat batas keputusan non-linear

Contoh: Menyelesaikan XOR dengan MLP

Arsitektur: 2 input → 2 hidden → 1 output

Hidden layer: Membuat 2 linear separators

Output layer: Mengkombinasikan hasil untuk XOR

Aktivasi: Sigmoid pada semua layer

Algoritma Backpropagation

Backpropagation adalah algoritma untuk **melatih MLP** dengan menghitung gradien error dari output ke input.

Konsep Dasar:

Forward pass: Hitung prediksi dari input ke output

Calculate error: Bandingkan prediksi dengan target

Backward pass: Propagasi error dari output ke input

Update weights: Adjust parameter berdasarkan gradien

Rumus Forward Pass:

Hidden Layer: $h = \sigma(W_1 \cdot x + b_1)$

Output Layer: $y = \sigma(W_2 \cdot h + b_2)$

Loss Function (Mean Squared Error):

$$L = \frac{1}{2} \sum (y_i - t_i)^2$$

Chain Rule untuk Gradien:

$$\partial L / \partial w = \partial L / \partial y \times \partial y / \partial \text{net} \times \partial \text{net} / \partial w$$

Algoritma Backpropagation

Rumus Forward Pass:

Hidden Layer: $h = \sigma(W_1 \cdot x + b_1)$

Output Layer: $y = \sigma(W_2 \cdot h + b_2)$

Loss Function (Mean Squared Error):

$$L = \frac{1}{2} \sum (y_i - t_i)^2$$

Chain Rule untuk Gradien:

$$\partial L / \partial w = \partial L / \partial y \times \partial y / \partial \text{net} \times \partial \text{net} / \partial w$$

Langkah-langkah backpropagation

- Initialize weights secara random
- Forward pass - hitung output untuk semua layer
- Hitung error di output layer
- Backward pass - propagasi error ke hidden layers
- Update semua weights dan biases
- Ulangi untuk semua training data

Detail Matematis Backpropagation

Hidden Layer:

$$\begin{aligned} \text{net_h} &= \sum (w_{ih} \times x_i) + b_h \\ \text{out_h} &= \sigma(\text{net_h}) \end{aligned}$$

Output Layer:

$$\begin{aligned} \text{net_o} &= \sum (w_{ho} \times \text{out_h}) + b_o \\ \text{out_o} &= \sigma(\text{net_o}) \end{aligned}$$

Backward pass menghitung error untuk setiap layer:

Output Layer Error:

$$\delta_o = (\text{target} - \text{output}) \times \sigma'(\text{net_o})$$

Hidden Layer Error:

$$\delta_h = \sigma'(\text{net_h}) \times \sum (\delta_o \times w_{ho})$$

Weight Updates:

$$\begin{aligned} \Delta w &= \eta \times \delta \times \text{input} \\ w_{\text{new}} &= w_{\text{old}} + \Delta w \end{aligned}$$

1

- Inisialisasi bobot (ambil bobot awal dengan nilai random yang cukup kecil).
- Tetapkan: Maksimum Epoch, Target Error, dan Learning Rate (α).
- Inisialisasi: Epoch = 0.
- Kerjakan langkah-langkah berikut selama (Epoch < Max Epoch) dan (MSE < Target Error):

1. Epoch = Epoch + 1

2. Untuk tiap-tiap pasangan elemen yang akan dilakukan pembelajaran, kerjakan:

Feedforward:

a. Tiap-tiap unit input ($X_i, i = 1, 2, 3, \dots, n$) menerima sinyal x_i dan meneruskan sinyal tersebut ke semua unit pada lapisan yang ada di atasnya (lapisan tersembunyi).

b. Tiap-tiap unit pada suatu lapisan tersembunyi ($Z_i, j = 1, 2, 3, \dots, p$) menjumlahkan sinyal-sinyal input terbobot:

$$z_{in_j} = b1_j + \sum_{i=1}^n x_i v_{ij}$$

gunakan fungsi aktivasi untuk menghitung sinyal outputnya:

$$z_j = f(z_{in_j})$$

dan kirimkan sinyal tersebut ke semua unit di lapisan atasnya (unit-unit output).

c. Tiap-tiap unit output ($Y_k, k = 1, 2, 3, \dots, m$) menjumlahkan sinyal-sinyal input terbobot.

$$y_{in_k} = b2_k + \sum_{i=1}^p z_i w_{jk}$$

gunakan fungsi aktivasi untuk menghitung sinyal outputnya:

$$y_k = f(y_{in_k})$$

dan kirimkan sinyal tersebut ke semua unit di lapisan atasnya (unit-unit output).

Detail Matematis Backpropagation

Hidden Layer:

$$\text{net_h} = \sum(w_{ih} \times x_i) + b_h$$

$$\text{out_h} = \sigma(\text{net_h})$$

Output Layer:

$$\text{net_o} = \sum(w_{ho} \times \text{out_h}) + b_o$$

$$\text{out_o} = \sigma(\text{net_o})$$

Backward pass menghitung error untuk setiap layer:

Output Layer Error:

$$\delta_o = (\text{target} - \text{output}) \times \sigma'(\text{net_o})$$

Hidden Layer Error:

$$\delta_h = \sigma'(\text{net_h}) \times \sum(\delta_o \times w_{ho})$$

Weight Updates:

$$\Delta w = \eta \times \delta \times \text{input}$$

$$w_{\text{new}} = w_{\text{old}} + \Delta w$$

2

- Kerjakan langkah-langkah berikut selama (Epo $<$ Max Epoch) dan (MSE $<$ Target Error):
 2. Untuk tiap-tiap pasangan elemen yang akan dilakukan pembelajaran, kerjakan:

Backpropagation:

- d. Tiap-tiap unit output ($Y_k, k = 1, 2, 3, \dots, m$) menerima target pola yang berhubungan dengan pola input pembelajaran, hitung informasi errornya:

$$\delta 2_k = (t_k - y_k) f'(y_{in_k})$$

$$\varphi 2_{jk} = \delta_k z_j$$

$$\beta 2_k = \delta_k$$

kemudian hitung koreksi bobot (yang nantinya akan digunakan untuk memperbaiki nilai w_{jk}):

$$\Delta w_{jk} = \alpha \varphi_{jk}$$

hitung juga koreksi bobot (yang nantinya akan digunakan untuk memperbaiki nilai $b 2_k$):

$$\Delta b 2_k = \alpha \beta_k$$

Langkah (d) ini juga dilakukan sebanyak jumlah lapisan tersembunyi, yaitu menghitung informasi error dari suatu lapisan tersembunyi ke lapisan tersembunyi sebelumnya.

Detail Matematis Backpropagation

Hidden Layer:

$$\text{net_h} = \sum (w_{ih} \times x_i) + b_h$$
$$\text{out_h} = \sigma(\text{net_h})$$

Output Layer:

$$\text{net_o} = \sum (w_{ho} \times \text{out_h}) + b_o$$
$$\text{out_o} = \sigma(\text{net_o})$$

Backward pass menghitung error untuk setiap layer:

Output Layer Error:

$$\delta_o = (\text{target} - \text{output}) \times \sigma'(\text{net_o})$$

Hidden Layer Error:

$$\delta_h = \sigma'(\text{net_h}) \times \sum (\delta_o \times w_{ho})$$

Weight Updates:

$$\Delta w = \eta \times \delta \times \text{input}$$
$$w_{\text{new}} = w_{\text{old}} + \Delta w$$

3

- Kerjakan langkah-langkah berikut selama (Epoch < Max Epoch) dan (MSE < Target Error):

2. Untuk tiap-tiap pasangan elemen yang akan dilakukan pembelajaran, kerjakan:

Backpropagation:

- e. Tiap-tiap unit tersembunyi ($Z_j, j = 1, 2, 3, \dots, p$) menjumlahkan delta inputnya (dari unit-unit yang berada pada lapisan di atasnya):

$$\delta_{in_j} = \sum_{k=1}^m \delta_k w_{jk}$$

kalikan nilai ini dengan turunan dari fungsi aktivasinya untuk menghitung informasi error:

$$\delta 1_j = \delta_{in_j} f'(z_{in_j})$$

$$\phi 1_{ij} = \delta_j x_j$$

$$\beta 1_j = \delta 1_j$$

kemudian hitung koreksi bobot (yang nantinya akan digunakan untuk memperbaiki nilai v_{ij}):

$$\Delta v_{ij} = \alpha \phi 1_{ij}$$

hitung juga koreksi bias (yang nantinya akan digunakan untuk memperbaiki nilai $b1_j$):

$$\Delta b1_j = \alpha \phi 1_j$$

- f. Tiap-tiap unit output ($Y_k, k = 1, 2, 3, \dots, m$) memperbaiki bias dan bobotnya ($j = 0, 1, 2, \dots, p$):

$$w_{jk}(\text{baru}) = w_{jk}(\text{lama}) + \Delta w_{jk}$$

$$b2_k(\text{baru}) = b2_k(\text{lama}) + \Delta b2_k$$

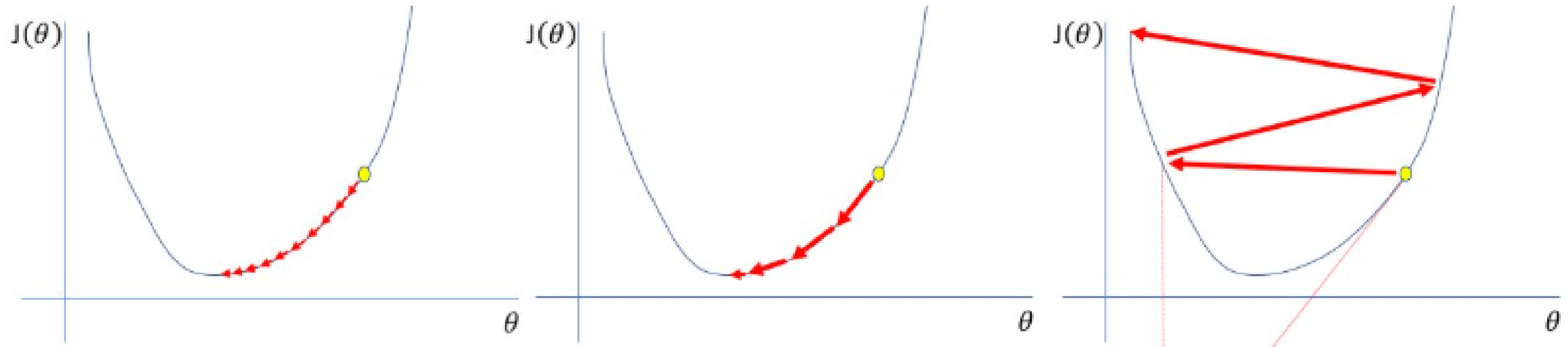
Tiap-tiap unit tersembunyi ($Z_j, j = 1, 2, 3, \dots, p$) memperbaiki bias dan bobotnya ($i = 0, 1, 2, \dots, n$):

$$v_{ij}(\text{baru}) = v_{ij}(\text{lama}) + \Delta v_{ij}$$

$$b1_j(\text{baru}) = b1_j(\text{lama}) + \Delta b1_j$$

3. Hitung MSE

Backpropagation – Learning Rate



Learning rate yang terlalu kecil akan menyebabkan proses pelatihan menjadi lebih lama untuk mencapai konvergen.

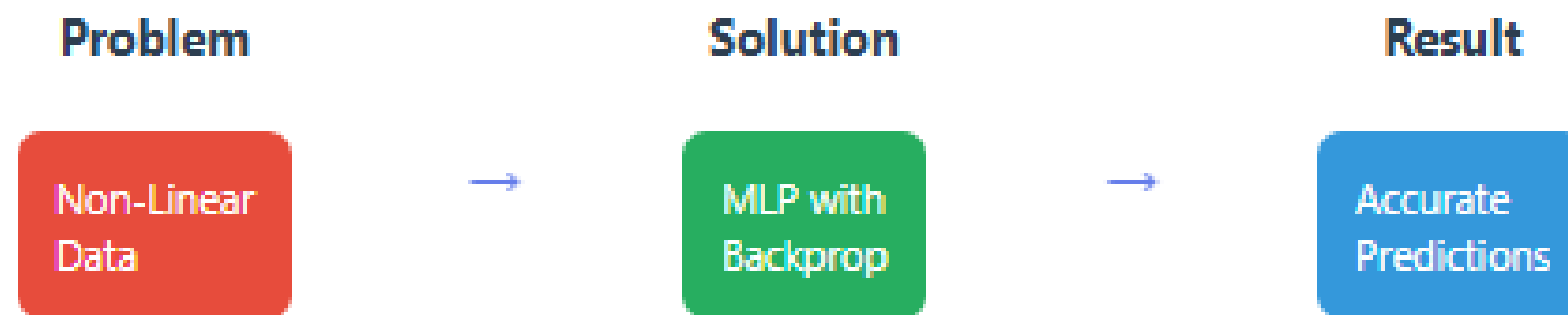
Learning rate yang optimal adalah jika step-nya tidak terlalu besar dan juga tidak terlalu kecil sehingga lebih cepat mencapai konvergen.

Learning rate yang terlalu besar berpotensi menyebabkan divergen, sulit mencapai titik optimal atau bahkan tidak pernah mencapai konvergen.

Ringkasan: Multi-Layer Perceptron

Key Points:

- MLP dapat menyelesaikan masalah non-linear yang tidak bisa diselesaikan perceptron tunggal
- Hidden layers + activation functions memberikan kemampuan non-linear
- Backpropagation adalah algoritma training utama untuk MLP
- Chain rule memungkinkan propagasi error dari output ke input



MLP Success Formula:
Multiple Layers + Non-linear Activation + Backpropagation
= Universal Function Approximator

Practice

Bandingkan performa MLP dengan menggunakan 3 fungsi aktivasi:

1. Sigmoid
2. Tanh
3. ReLU

Dengan menggunakan dataset iris yang ada di kaggle.

Tampilkan table perbandingan performa

Setelah di proses, fungsi aktivasi apa yang terbaik untuk dataset iris tersebut.

Reference

1. <https://medium.com/@cmukesh8688/activation-functions-sigmoid-tanh-relu-leaky-relu-softmax-50d3778dcea5>
2. https://www.researchgate.net/publication/341310767_Machine_Learning_for_Materials_Developments_in_Metals_Additive_Manufacturing

THANK YOU!